IRWA Final Project 2026 – Part 1

Text Processing and Exploratory Data Analysis

Antonio Javier Ortiz Gil (u233542), Juan Antonio Oliver Blanes (u232701) and Aitor Bolige Cubas (u232752)

GitHub: https://github.com/ToniOrtizGil/irwa-search-engine-2026 ; Tag: IRWA-2026-part-1

Code. `myapp/preprocessing/text_processing.py` (text pre-processing), `myapp/preprocessing/fields.py` (fields), `project_progress/part_1/run_preprocessing.py` (processes the corpus) and `project_progress/part_1/part1_eda.ipynb` (pre-processing + EDA; it generates all the numbers and figures of this report). The README explains how to run it.

The collection. N = 28,080 documents with 17 fields and no repeated `pid`s. `description` is empty in 39.7% of the documents, `average_rating` in 8.1%, `brand` in 7.2% and `seller` in 5.9%. The titles are very repetitive: only 5,953 different titles.

1 Part 1: Data preparation

1.1 Pre-processing of title and description

As in the lab, the function build_terms() transforms a text into a list of index terms. We also use the function for documents and queries, so both are represented with the same terms. Our normalization policy is:

1. **Lowercase**, so “Cotton” and “cotton” are the same term.
2. **Tokenization and removing punctuation marks**: we split on spaces and punctuation. As seen in class, the tokenizer must consider hyphens, apostrophes and non-alpha characters. Here this is important because many descriptions have no space after a dot or a comma. With this we also remove strange characters from the crawling (non breaking spaces, “Â”, “â”).
3. **Removing stop words** (“stopping”) with the NLTK English list. We checked that no query word (women, men, full, sleeve, slim…) is in the list.
4. **Stemming** with the Porter stemmer, as in the lab (jeans → jean, sleeve → sleev, printed → print). Queries are short and users write singular or plural indistinctly (“jean”/“jeans”), when we group words with a common stem helps us to find more relevant documents. As seen in class, it is usually effective but not for all queries: a few different words can get the same stem.

Extra steps:

- **“t-shirt”** → **tshirt**, before splitting. Otherwise we get the tokens “t” and “shirt”, and “shirt” would also match normal shirts.
- **Gender words**: woman, womens → women and man, mens → men. Gender is key in fashion queries and the stemmer does not group *woman* and *women*.
- **Number elimination**: numbers in the text are prices, percentages or sizes; this information is kept in separate numeric fields.
- **Removing one-character tokens**, like the “s” left by “women's”.

Limitations: Removing numbers loses some information in the text (“pack of 2”), but for most products the number of items is also in the attribute “Pack of”. We use the NLTK stop word list without changes, because no fashion word is in it. Lowercasing also changes brand names, but brands can be filtered with the brand field.

Example: women full sleeve sweatshirt cotton → [women, full, sleev, sweatshirt, cotton]. Table 1 shows that every step reduces the vocabulary, from 13,521 to 4,716 terms.

Step (title + description of all documents)	|V|	Tokens
0. raw text (split on spaces)	13,521	1,017,425
1. lowercase + split on punctuation	6,497	1,031,686
2. remove numbers	6,326	1,016,181

3. remove stop words	6,198	689,771
4. remove one-character tokens	6,181	687,525
5. stemming (Porter)	4,716	687,525

Table 1: Vocabulary size and number of tokens after each step.

1.2 Information returned for each document

For each document we store a dictionary meta with the fields that the results must return: pid, title, description, brand, category, sub_category, product_details, seller, out_of_stock, selling_price, discount, actual_price, average_rating, url. They are kept as they are, except the numeric fields (converted to numbers) and product_details (converted from a list of small dictionaries into one dictionary). Missing values are stored as empty / None, so the result can show “n/a”. Next to meta we store the index terms of each field, so we search with the terms and show the original text. In addition, pid is never modified. Everything is saved in data/processed/processed_corpus.pkl.

1.3 Category, sub_category, brand, product_details and seller

Approach	Pros	Cons
Merge into one text field	Simple, works directly with the inverted index of the lab. A query term is found in any field.	All fields count the same. Fields with almost the same value in all documents only add terms with very low idf.
Separate fields in the inverted index	Each field can have a different weight in the ranking, and we can filter by field (e.g. by brand).	More complex. Queries don't say which field they refer to, so the scores of the fields must be combined.
Both	We can index the combined field in Part 2 and add weights or filters in Part 3.	More memory, we must choose which fields go into the combined one.

Table 2: Pros and cons of each approach.

Distinctiveness: As seen in class, rare terms are more informative than frequent terms: a value that appears in almost all documents has df ≈ N and idf = log(N/df) ≈ 0, so it does not help us to rank. We measured it for each field:

- **category**: 4 values; 96.6% of the documents are “Clothing and Accessories” (idf = 0.035). It does not help to rank, so it is only kept for filtering
- **sub_category**: 24 values (Topwear 54.5%, idf = 0.61; Bottomwear 13.1%; Winter Wear 8.9%). It separates, for example, jeans from sweatshirts, so it goes into the combined field. 123 documents have the brand glued to the value.
- **brand**: 324 brands, but many names are cut in the dataset (ej:“REEB”), so users will probably not type them like that. We add it to the combined field and keep it separately for filtering.
- **seller**: 534 sellers. It describes the shop, not the product, and no query term appears in it, so it is only kept for filtering.
- **product_details**: 127 attributes, about 13 per document. Fabric (92%), Pattern (92%), Sleeve (67%), Ideal For (61%), Fit (60%) and Color (51%) contain exactly the words of fashion queries, and 40% of the descriptions are empty, so it is very important. We index only the *values* and skip useless attributes: Style Code (23,263 different codes), model name/number, Country of Origin (98.9% “India”), Fabric Care (washing instructions), pack, size and values like “Yes”/“No”.

Decision: both. We store the terms of each field separately and also we build a combined field search_tokens = title + description + brand + sub_category + product_details values (43 terms per document on average). Category and seller are only used for filtering. We chose it because the combined field works directly with the inverted index and TF-IDF that we will implement in the incoming parts. The extra memory is small (28,080 documents).

1.4 out_of_stock, selling_price, discount, actual_price and average_rating

They are not indexed as textual terms. As we have seen in class, they are like the fields of a database record: they have a well-defined meaning and are compared directly with the query (e.g. price < 500). Some terms like “921” and “922” would be unrelated terms and we could not search by price range. So we convert them to numbers: “2,999” → 2999.0, “69% off” → 69.0, “3.9” → 3.9, and out_of_stock stays as a boolean. Missing values become None, we not put 0 because a product without rating is not a product with a bad rating. The conversion works for 99.99% of the selling prices, 97.2% of the actual prices, 97.0% of the discounts and 92.0% of the ratings. We also checked that the selling price is never higher than the actual price and that the discount matches both prices. In next steps they will be used for filtering and in the ranking.

1.5 Validation labels

validation_labels.csv contains relevance judgments: query 1 (“women full sleeve sweatshirt cotton”) has 13 relevant and 7 non-relevant documents and query 2 (“men slim jeans blue”) has 10 and 10. We checked in which fields the query terms appear:

- The title separates relevant from non-relevant documents best: for query 1, full and sleev are in 100% of the relevant titles but only in 43% of the non-relevant ones.
- cotton never appears in the titles, but it is in 85% of the relevant documents through product_details. Without this field we could not match the material.
- The description does not help because the query terms appear about the same in relevant and non-relevant documents.

Exact matching of words is not enough: documents with the same title get different judgments (“Full Sleeve Printed Women Sweatshirt” in query 1, “Slim Women Blue Jeans” in query 2), and three women's jeans are relevant for a query about men's jeans.

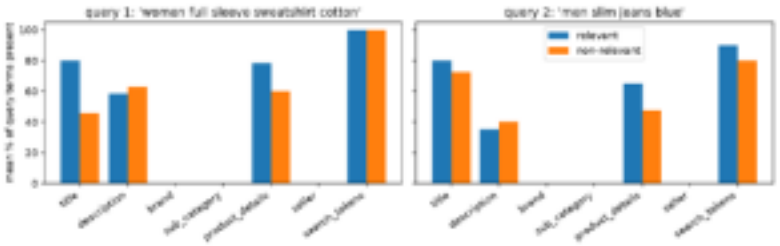


Figure 1: Average % of the query terms in each field, relevant vs non-relevant documents.

2 Part 2: Exploratory data analysis

2.1 Word counting distribution and average sentence length

Titles have 6.5 words on average (median 6, max 38). Descriptions have 29.8 words on average, but the median is only 10 because 40% are empty, and the longest has 623 words. Splitting the descriptions on “.”, “!” and “?”, they have 3.3 sentences on average with 14.8 words per sentence.

2.2 Vocabulary size, Zipf's law and Heaps' law

After pre-processing, |V| is 649 terms in titles, 4,557 in descriptions, 2,268 in product details and 5,733 in the combined field; 22.3% of the terms occur only once. The small title vocabulary confirms that titles follow a template (pattern + gender + type + color). The term frequencies follow Zipf's law ($r \cdot p(r) \approx c$): $r \cdot p(r)$ is 0.185 at r = 10 and 0.163 at r = 100, with problems at high and low frequencies. For Heaps' law ($v = k \cdot n^{\beta}$) we got k = 2.68 and $\beta = 0.55$: β is close to the typical 0.5, but k is below the typical range (10–100) because the texts are very repetitive and we count stemmed terms.

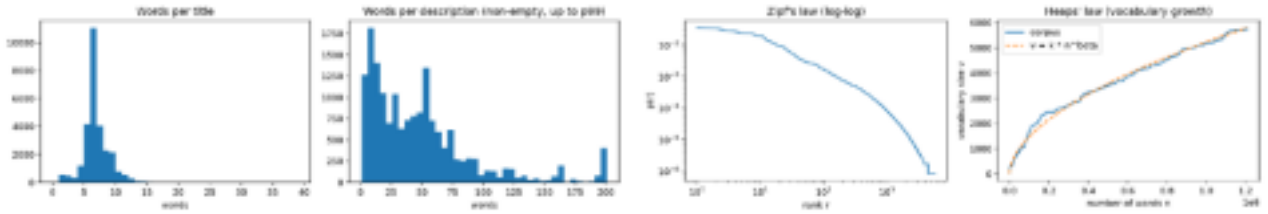


Figure 2: Words per title and per description. **Figure 3:** Zipf's law and Heaps' law.

2.3 Word clouds for the most frequent words

We built them without stemming to read the words better. The most frequent title terms are women (13,146), men (13,060), tshirt, neck, solid and round, most products are t-shirts and there are as many products for men as for women. Descriptions are full of advertising words (cotton, wear, comfort, look, casual) that appear in thousands of documents, so their idf will be low.



Figure 4: Word clouds of titles and descriptions.

2.4 Ranking of products by rating, price and discount

Ratings are mostly between 3.2 and 4.1 (average 3.63) and 329 products have 5.0, so we break ties by discount (Table 3). We don't know the number of reviews, so these 5.0 ratings may come from very few reviews. Prices: median 545 INR, max 7,999; the actual price is about twice higher (median 1,199) because the average discount is 50% (max 87%) (Figure 5). The ratings of the most expensive products are not especially high (2.6–4.5 in the top 10).

Ranking	Product (brand)	Price	Discount	Rating
Rating	Men Solid Cotton Blend Pathani Kurta (Purple Sta)	449	82%	5.0
	Printed Men Boxer (SayItLo)	249	78%	5.0
Price	Solid Women Track Suit (REEB)	7,999	20%	2.7
	Checkered Single Breasted Party Women Blazer (True Bl)	7,998	–	2.6
Discount	Printed Women Round Neck White T-Shirt (Jack Roy)	128	87%	3.0
	Printed Women Round Neck Grey T-Shirt (Jack Roy)	136	86%	4.1

Table 3: Top 2 products of each ranking (top 10 in the notebook).

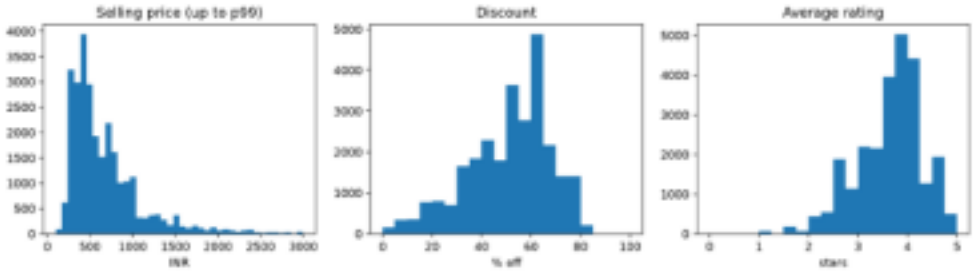


Figure 5: Selling price, discount and rating.

2.5 out_of_stock distribution

Only **5.9%** of the products (1,644) are out of stock (Figure 6), so hiding them will rarely empty a result page. The sub_categories with more products out of stock are Clothing Accessories (16.9%) and Tracksuits (14.4%).

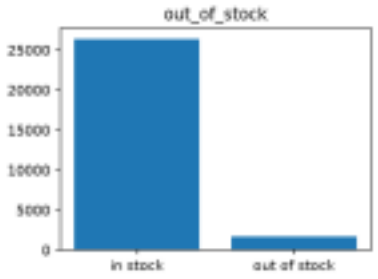


Figure 6: out_of_stock distribution.

2.6 Top sellers and brands

The brands with more products are ECKO Unl (951), Free Authority (860), ARBO (806), REEB (802) and Pu (798); the sellers are RetailNet (1,411), SandSMarketing (887) and BioworldMerchandising (842). Among brands with at least 20 products to avoid the ones with only 1 or 2 products, the best rated are VARTe (5.0), SORA (4.61) and PixF (4.46).

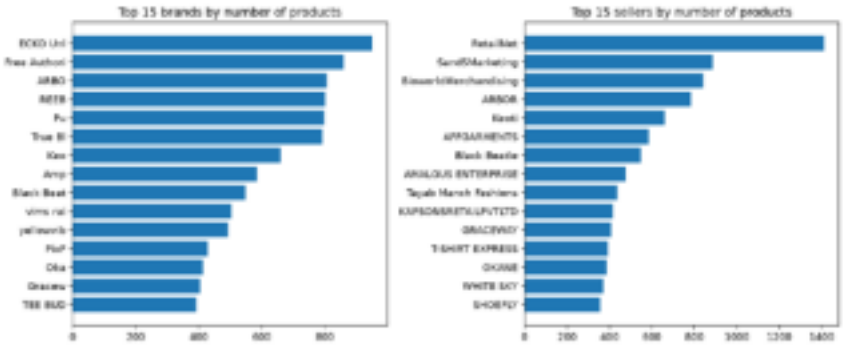


Figure 7: Brands and sellers with more products.

2.7 Entity recognition

As we have seen in class, information extraction can identify classes of index terms with a named entity recognizer (people, locations, companies…). We also used the spaCy recognizer in the descriptions. The most common classes are ORG (14,330), PERSON (9,667), PERCENT (5,483), DATE (5,425) and GPE (4,157), but some entities are wrong: “Cotton” and “kurta” are detected as locations, “Jeans” as a nationality and “Track Pant” as a person. Only the percentages (“100%” cotton) and real places (“India”) are correct. The useful information (brand, fabric, color) is already in brand and product_details, so we do not use the entities in the index.

2.8 Other findings

Almost in every women's products, the URL don’t contain “women” (12,980 of 13,041 titles with “Women”, 12,825 of them contain “-men”) while the title never contradicts “Ideal For”, so we trust the title and product details and we don’t index the URL.

Use of AI tools

We used Claude (Anthropic) to brainstorm the pre-processing strategy and to help write and debug the code. All the code was reviewed and run by the group, all numbers come from our own execution of the notebook, and the decisions and explanations were checked and edited by the group members, who take full responsibility for the content.